

Project #3: End-to-End ML/DL Project

Salient Object Detection with a U-Net CNN

Prepared by: Dren Rrustemi

This report documents the complete implementation of a Salient Object Detection (SOD) system. The goal was to build an end-to-end deep learning pipeline that receives an RGB image and predicts a binary saliency mask that highlights the most visually important object or region in the image.

1. Project Overview

The project was implemented in TensorFlow/Keras using Google Colab with GPU support. The solution covers dataset loading, preprocessing, augmentation, CNN model design from scratch, training, validation, evaluation, visualization, and a simple demo workflow for uploading a new image and viewing the predicted mask.

Item	Final implementation
Task	Salient Object Detection / image segmentation
Dataset	ECSSD public dataset
Input size	128 x 128 RGB images
Framework	TensorFlow 2.20.0 / Keras
Environment	Google Colab with GPU
Train / Val / Test split	700 / 150 / 150 images = 70% / 15% / 15%
Models compared	Baseline U-Net vs Improved U-Net
Saved output	baseline_unet.keras and improved_unet.keras saved to Google Drive

2. Dataset and Preparation

For the final version, the ECSSD dataset was used. It is suitable for salient object detection because each input image has a matching ground-truth saliency mask. The notebook verifies that both the image folder and mask folder exist, then pairs files by matching image stems with mask filenames. In the final run, 1000 image/mask pairs were detected and used.

The dataset was split using sklearn train_test_split with a fixed random seed to make the split reproducible. First, 70% of the pairs were assigned to training and 30% were held out temporarily. The temporary set was then split equally into validation and test sets, giving 15% validation and 15% test data. This matches the assignment requirement exactly.

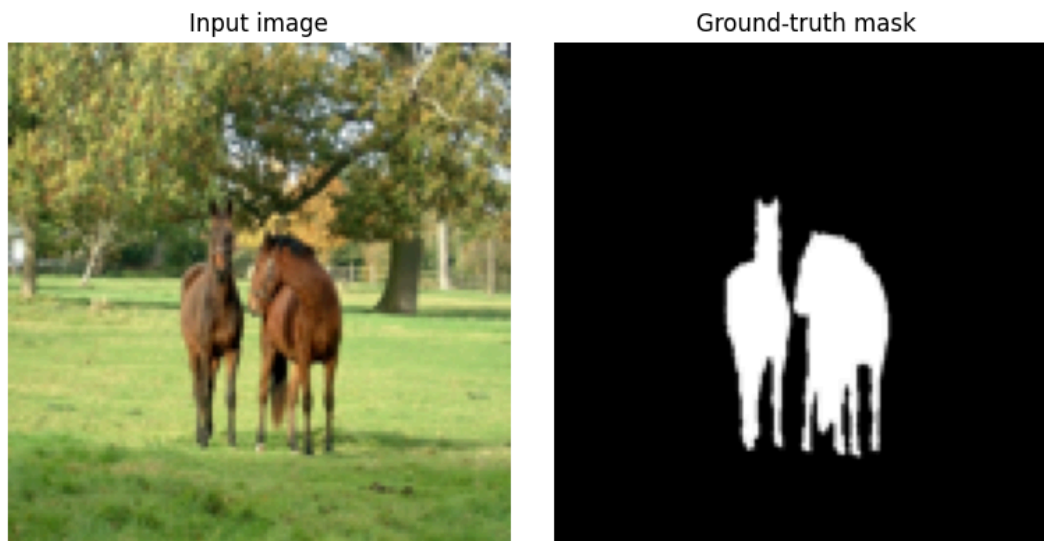


Figure 1. Example ECSSD input image and matching ground-truth saliency mask.

3. Preprocessing and Data Augmentation

Each image is loaded from disk, decoded, resized to 128 x 128, converted to float32, and normalized to the [0, 1] range. Each mask is decoded in grayscale, resized using nearest-neighbor interpolation to preserve class boundaries, normalized, and thresholded so that the final mask is binary.

Augmentation is applied only to the training dataset. The final notebook uses horizontal flipping, random brightness variation, and a crop-like random zoom process. These augmentations help the model see more variation during training and reduce overfitting while keeping the segmentation target aligned with the input image.

Preprocessing summary: resize to 128 x 128, normalize RGB values, binarize masks, batch the data, cache/prefetch for performance, and shuffle training samples using a fixed seed.

4. Model Architecture

The CNN was implemented from scratch as a U-Net style encoder-decoder model. U-Net is a good fit for saliency detection because it combines high-level semantic information from the encoder with spatial detail from earlier layers through skip connections. The output layer uses a 1-channel sigmoid activation so every pixel receives a probability of belonging to the salient foreground.

Part	Implementation	Purpose
Encoder	Repeated Conv2D + BatchNorm + ReLU blocks followed by MaxPooling	Extract increasingly abstract image features
Bottleneck	Deep convolutional block at the smallest spatial resolution	Capture the strongest semantic representation
Decoder	Conv2DTranspose upsampling + skip connections + conv blocks	Recover spatial resolution for pixel-wise prediction
Output	1 x 1 Conv2D with sigmoid activation	Produce a binary saliency probability map

Two model configurations were compared. The baseline model used base_filters=8 and no dropout. The improved model used base_filters=32 and dropout_rate=0.05. Batch normalization is included inside the convolution blocks in both models.

5. Training Logic

The training loop was handled through Keras `model.fit`, but the notebook still defines the full training process: model compilation, optimizer, loss function, metrics, validation at the end of every epoch, model checkpoints, early stopping, learning-rate reduction, and CSV logging.

The optimizer is Adam with a learning rate of $1e-4$. The loss function used in the notebook is Binary Cross-Entropy combined with Dice Loss. This is a common segmentation loss because Binary Cross-Entropy helps with per-pixel classification while Dice Loss directly encourages overlap between the predicted mask and the ground-truth mask.

Training component	Used in project
Optimizer	Adam, learning_rate = $1e-4$
Loss	Binary Cross-Entropy + Dice Loss
Metrics during training	Accuracy, Precision, Recall, Dice coefficient, IoU
Callbacks	ModelCheckpoint, EarlyStopping, ReduceLROnPlateau, CSVLogger
Epochs	15 maximum, with early stopping enabled

6. Experiments and Improvements

The assignment required at least two changes after the baseline model. The final implementation satisfies this by increasing the model capacity and adding dropout regularization. Data augmentation was also included in the training dataset. The comparison is therefore between a lightweight baseline and a stronger improved model.

Experiment	Base filters	Dropout	Main idea
Baseline U-Net	8	0.00	Fast, smaller model for reference
Improved U-Net	32	0.05	More feature capacity plus regularization

The improved model has more capacity, so it can learn more detailed segmentation patterns. The tradeoff is that inference becomes slower, because the model contains more filters and therefore more computation per image.

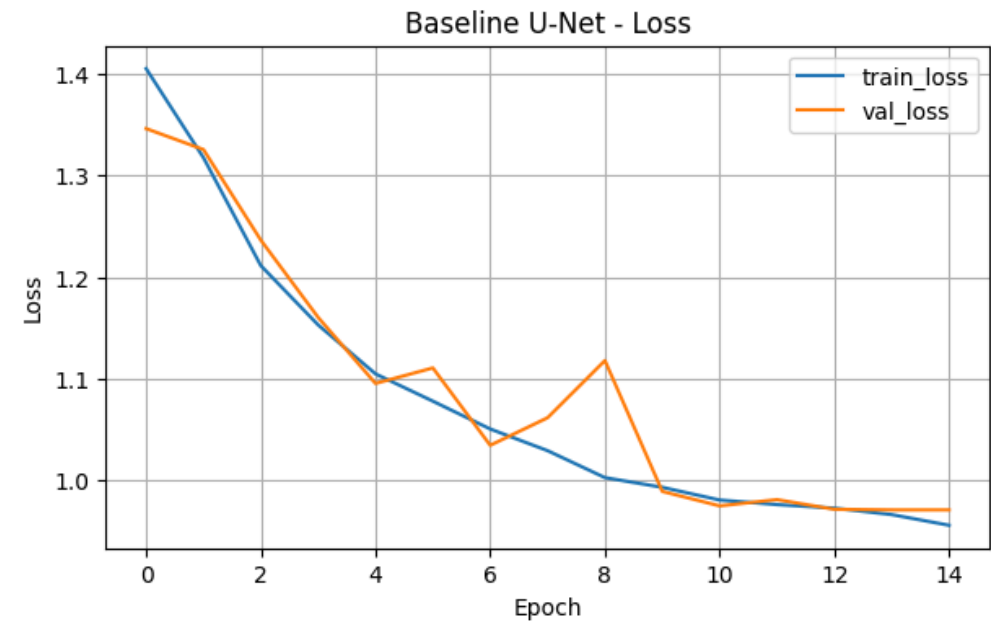


Figure 2. Baseline U-Net loss curve.

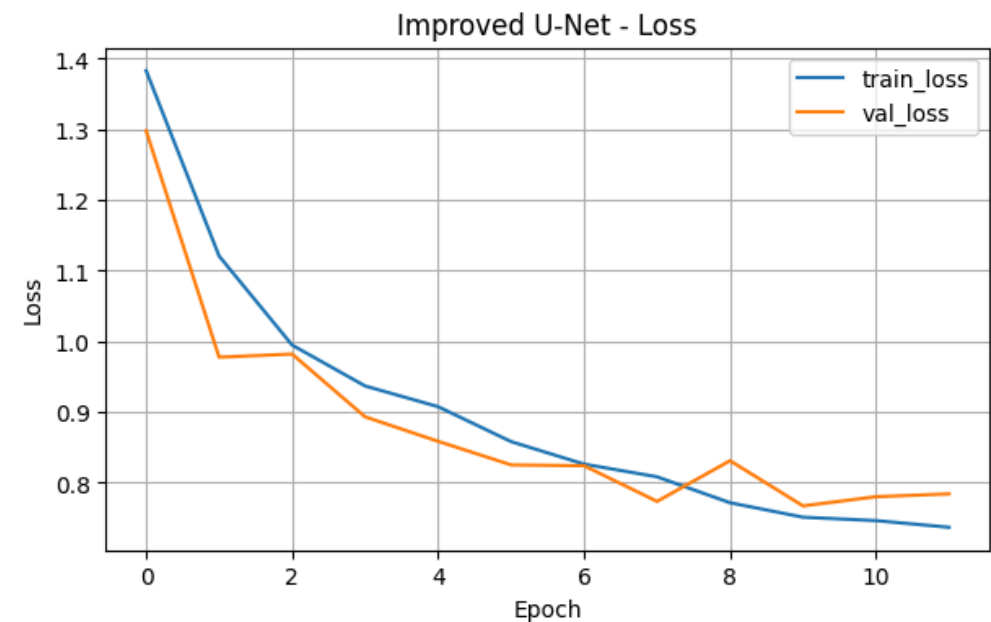


Figure 3. Improved U-Net loss curve.

7. Final Evaluation Results

After training, both models were evaluated on the independent test set. The final test metrics include IoU, Precision, Recall, F1-score, Dice, and average inference time per image. These metrics measure both segmentation quality and practical demo speed.

Model	IoU	Precision	Recall	F1	Dice	Avg sec/image
Baseline U-Net	0.4805	0.5534	0.7848	0.6491	0.6491	0.0185
Improved U-Net	0.5294	0.6122	0.7965	0.6923	0.6923	0.0379

The improved model achieved better IoU, Precision, Recall, F1-score, and Dice score than the baseline. Its IoU increased from 0.4805 to 0.5294, and its F1/Dice score increased from 0.6491 to 0.6923. This confirms that the improvements helped the model produce better saliency masks. The only negative tradeoff is speed: average inference time increased from about 0.0185 seconds per image to about 0.0379 seconds per image.

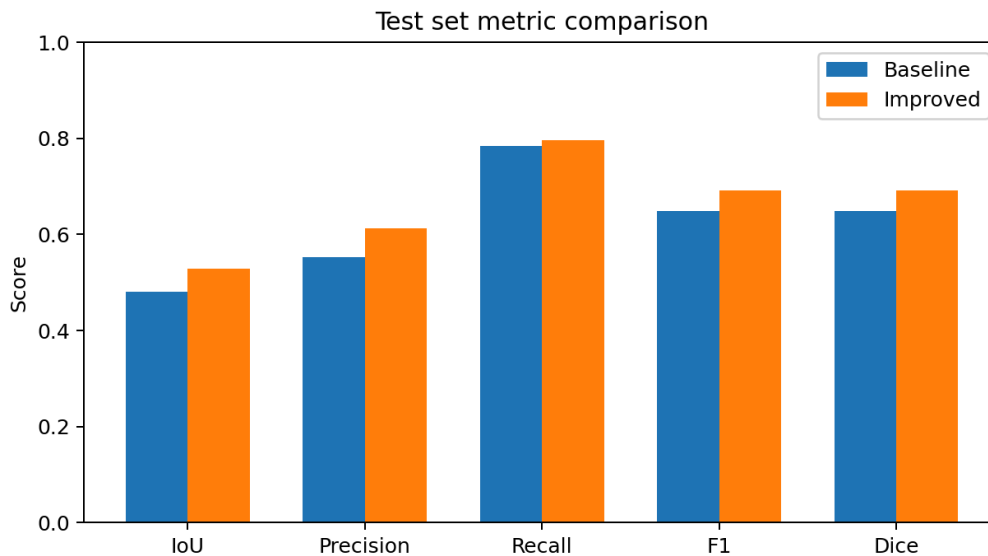


Figure 4. Final metric comparison between baseline and improved model.

8. Visualization and Demo

The notebook includes visual outputs for model interpretation. For test samples, it displays the input image, the ground-truth mask, the predicted mask, and an overlay of the prediction on top of the original image. This is important because segmentation metrics alone do not show where the model succeeds or fails visually.



Figure 5. Prediction visualization: input, ground truth, predicted mask, and overlay.

A separate single-image demo cell was also added. In the demo, the user uploads an image in Colab, the improved model predicts a mask, and the notebook displays the input, the predicted mask, and the overlay. The final demo output reported an inference time of 0.0951 seconds for the uploaded example image.

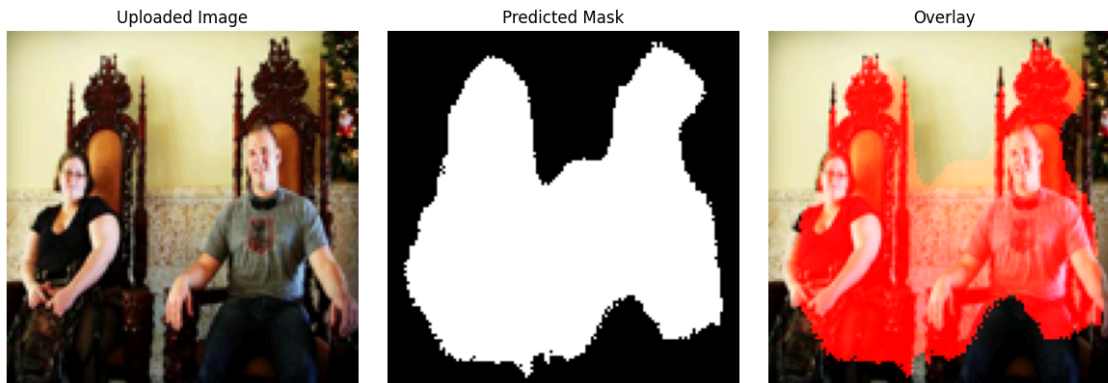


Figure 6. Single-image demo output with predicted saliency mask and overlay.

9. Requirement Coverage

The table below summarizes how the final implementation matches the project requirements.

Requirement	Status	Evidence in final notebook
Public SOD dataset	Completed	ECSSD dataset with 1000 paired images/masks
Resize and normalize	Completed	Images resized to 128 x 128 and normalized to [0,1]
70/15/15 split	Completed	Train 700, validation 150, test 150
CNN from scratch	Completed	Custom U-Net implementation in Keras
Training and validation	Completed	model.fit with validation dataset and callbacks
Evaluation metrics	Completed	IoU, Precision, Recall, F1, Dice, inference time
Visualizations	Completed	Input, ground truth, predicted mask, overlay
Demo	Completed	Colab upload demo for a new image

10. Conclusion and Learnings

The final project successfully implements an end-to-end salient object detection pipeline. The model can load an image, preprocess it, predict a saliency mask, and visualize the result with an overlay. The improved U-Net performed better than the baseline on all quality metrics, showing that increasing model capacity and adding regularization improved segmentation performance.

The main learning from the experiments is that segmentation quality and inference speed must be balanced. The baseline was faster, but the improved model produced better masks. For a final presentation and demo, the improved model is the better choice because it gives stronger results while still keeping inference time low enough for interactive use.

Future improvements could include training for longer on a larger dataset, experimenting with stronger augmentations, tuning the segmentation threshold, and trying a more advanced loss such as Binary Cross-Entropy combined with IoU loss. However, the current solution already satisfies the core requirements of the assignment and provides a complete reportable deep learning workflow.